Who Waits Longer in Buffalo? Measuring Fairness in 311 Response Times Across Neighborhoods

Phase 2: Applying Machine Learning and Statistical Models

Jayda Johnson
Dept. of Computer Science and Engineering
University at Buffalo
Buffalo, NY, USA

Meena Kasina
Dept. of Computer Science and Engineering
University at Buffalo
Buffalo, NY, USA

Josh Hu
Dept. of Computer Science and Engineering
University at Buffalo
Buffalo, NY, USA

***Abstract*—To examine whether 311 response times are fair across Buffalo, we analyzed about 528,000 cleaned service requests from 2020 to 2026 using six different algorithms: Linear Regression, MapReduce, Logistic Regression, Naive Bayes, Random Forest, and k-Means. Linear regression and MapReduce algorithms were used to measure the typical wait times in each district and track how these differences changed over the years. We used logistic regression, Naive Bayes, and Random Forest to predict which requests would experience slow responses and identify the most important factors behind delays. Furthermore, k-Means clustering allowed us to group neighborhoods with similar service patterns. Across all our models, the type of request was the most important factor in determining slow service, while location was less significant. However, the North district and some neighborhoods on the East Side and in the Northeast still experienced slower response times. The gap between districts increased from 2020 to 2024, which was when Buffalo changed its 311 system, but then narrowed somewhat in 2025 and 2026.**

***Keywords*—311 service requests, machine learning, random forest, k-means, MapReduce, service equity**

I. Introduction

Buffalo residents use 311 service requests to report everyday non-emergency problems like potholes, missed trash pickups, and housing violations. In Phase 1, we merged the City of Buffalo's two 311 datasets [1], [2], cleaned them, and explored them, ending with about 528,000 requests from 2020 to 2026. In Phase 2, we applied six algorithms to answer the questions we set in Phase 1:

1) **Q1:** Which council districts and neighborhoods wait the longest for 311 service requests to be resolved?
2) **Q2:** Do response times differ by request type or department, and does that explain the gaps between areas?
3) **Q3:** Have these gaps gotten better or worse from 2020 to 2026?
4) **Q4:** Which neighborhoods share similar service patterns, and do the slowest-served areas have something in common?

We refined Q4 from Phase 1, which asked about neighborhoods after accounting for population. Answering that version would have required adding Census population data, so we reframed it around service patterns we can measure with our existing data. The project still uses live, externally hosted data loaded directly from the City of Buffalo Open Data Portal, accessed October 15 to 18, 2026, so row counts differ slightly between our notebooks depending on the day each was run.

II. Shared Setup

All three of our notebooks start by rerunning our full Phase 1 cleaning pipeline, then add the same set of features so every algorithm works from identical data:

- **resolutionDays:** days from Open Date to Closed Date (only for closed requests)
- **openYear, openMonth, season:** when the request was opened
- **afterSwitch:** whether the request was opened on or after May 11, 2024, when Buffalo switched 311 systems
- **isSlow:** whether a closed request took longer than the median for its request category

Defining "slow" relative to each request category matters for fairness: a pothole is only compared to other potholes, and a housing violation to other housing violations. This keeps one slow request type from making an entire district look slow. Because of this definition, about half of all closed requests are labeled slow. Only closed requests (about 514,000) have a resolution time, so all supervised models use closed requests only, with an 80% training and 20% testing split. All models were built with scikit-learn [3].

III. Linear Regression

A. Why We Chose It

Linear regression predicts a number, so it fits Q1 directly: how many days does a request take? Its coefficients let us compare districts while holding request type, month, and the 2024 system switch constant. Some districts simply receive more slow request types, like housing problems, so comparing raw averages would be unfair. Regression compares districts on the same kinds of requests.

B. Tuning and Training

The target was resolution days. The features were council district, request type (25 categories from Phase 1), opening month, and the afterSwitch flag, one-hot encoded with DELAWARE as the reference district, for 44 features in total. We trained on 411,469 requests and tested on 102,868. Because resolution times are heavily skewed (the median is about 4 days, but some requests take over 1,400 days), because a few requests

take over 1,400 days while most take about 4, we trained two versions of the model: one that predicts days directly, and one that predicts the log of days, which shrinks extreme values so they do not dominate, and then converts its predictions back to days. We compared both to a simple baseline that predicts the median wait time for every request.

C. Metrics

TABLE I. LINEAR REGRESSION TEST RESULTS

Model	R^2 (log scale)	MAE (days)
Baseline (median)	-0.012	13.56
Raw days	-0.143	15.54
Log days	0.290	12.30

The log model performed best, with the lowest mean absolute error (12.3 days) and an R^2 of 0.29 on the log scale. The raw-days model did worse than simply guessing the median, which confirms the log transform was necessary. An R^2 of 0.29 means the model explains less than a third of the variation in wait times; factors like crew availability, weather, and request severity are not in our data. Fitting separate models with one feature group at a time showed that request type alone reaches $R^2 = 0.277$, close to the full model, while district (0.002) and month (0.007) explain almost nothing on their own.

D. What It Tells Us

With request type, month, and the system switch held constant, most district differences are under 5% (Fig. 1). NORTH stands out at about 13.5% longer than DELAWARE, and SOUTH is about 5.8% shorter. UNIVERSITY has the largest raw median wait but is about 2.7% shorter than DELAWARE once request type is accounted for, so its long raw waits come mostly from the kinds of requests its residents make.

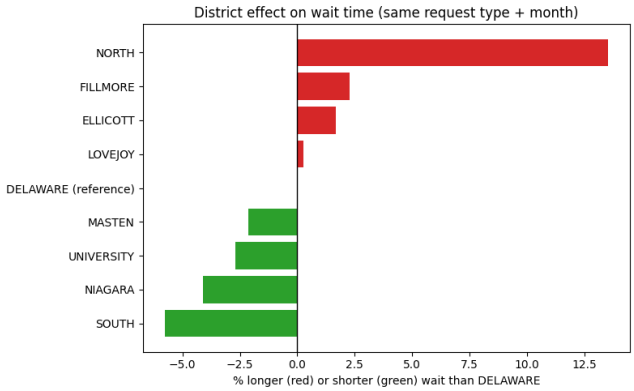


Fig. 1. District differences in predicted wait compared to DELAWARE, with request type, month, and system switch held constant.

IV. MAPREDUCE

A. Why We Chose It

To answer Q1 and Q3, we needed the median wait for every district in every year: 63 groups (9 districts × 7 years) from 514,337 closed requests. This is a natural MapReduce problem [4]: each request can be processed independently by a mapper, and a reducer combines each group. We used medians rather than averages because waits over 1,000 days would distort the mean.

B. How We Built It

We implemented map, shuffle, and reduce in plain Python, splitting the input into 4 chunks of about 128,585 requests to mirror how the work would be divided across 4 machines. The mapper emits (district, year) as the key and resolution days as the value; the shuffle groups all values with the same key; and the reducer sorts each group and returns its median. Because a median of chunk medians is not the true median, the shuffle sends every value for a key to the same reducer. We reran the job with (district, system period) as the key to compare before and after May 2024.

C. Metrics

All 63 medians matched the results from pandas groupby exactly, and the full job ran in 0.2 seconds. One caveat: open requests are excluded, and open rates were higher in 2024 (5.8%) and 2026 (8.4%) than in other years (under 2%). If long-waiting requests are more likely to still be open, recent medians are underestimated.

D. What It Tells Us

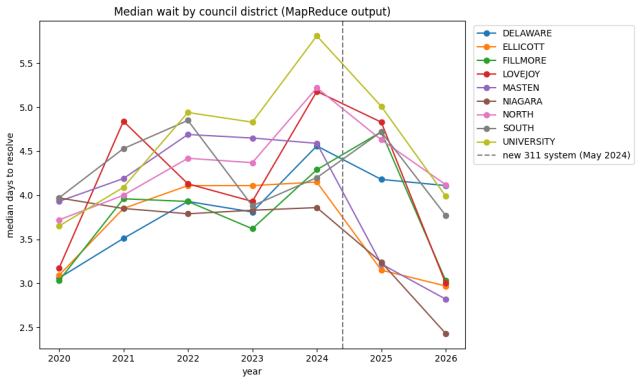


Fig. 2. Median resolution days by council district and year. The dashed line marks the May 2024 system switch.

TABLE II. SLOWEST AND FASTEST DISTRICT MEDIANS BY YEAR

Year	Slowest (days)	Fastest (days)	Gap
2020	NIAGARA (3.97)	FILLMORE (3.03)	0.94
2021	LOVEJOY (4.84)	DELAWARE (3.51)	1.33
2022	UNIVERSITY (4.94)	NIAGARA (3.79)	1.15
2023	UNIVERSITY (4.83)	FILLMORE (3.62)	1.21
2024	UNIVERSITY (5.81)	NIAGARA (3.86)	1.95
2025	UNIVERSITY (5.01)	ELLICOTT (3.15)	1.86
2026	NORTH (4.12)	NIAGARA (2.43)	1.69

Median waits rose in most districts from 2020 to 2024 (Fig. 2), and the gap between the slowest and fastest district roughly doubled, from 0.94 days in 2020 to 1.95 days in 2024, before easing to 1.69 days in 2026 (Table II). Comparing system periods, the gap was 0.71 days before May 2024 and 1.92 days after. Not every district got slower: NIAGARA and MASTEN had lower medians after the switch, while DELAWARE,

NORTH, and LOVEJOY rose by about a day. For Q3, the gaps grew over our study period, though we cannot attribute this to the system change alone, since the two systems may record closure times differently.

V. Logistic Regression

A. Why We Chose It

Logistic regression predicts a yes or no outcome, which fits Q2: will a request be slow (1) or not (0) compared to the median for its request type? It also gives a probability for each request, which lets us measure how well the model separates slow and fast requests.

B. Tuning and Training

We split the data 80/20 (random state 42). Because isSlow was originally computed on the full dataset, we recalculated each request type's median using only the training data, so no information from the test set leaked into the labels. Our first model used neighborhood, season, request type, and department, and reached 54.4% accuracy (ROC-AUC 0.567). Since that was barely better than guessing, we added more features: day of week, hour opened, month, a weekend flag, zip code, and latitude and longitude (filled with the median if missing, then standardized). Categorical features were one-hot encoded.

C. Metrics

The improved model reached 57.5% accuracy and a ROC-AUC of 0.605. For slow requests, precision was 0.58, recall 0.53, and F1 0.56; the model was better at catching fast requests (recall 0.62). The ROC curve (Fig. 3) sits only slightly above the random baseline.

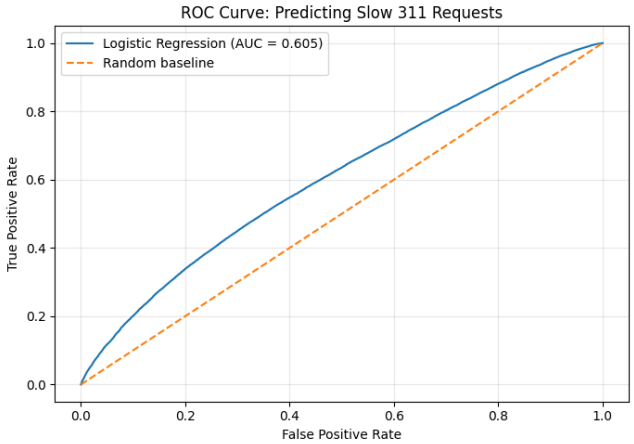


Fig. 3. ROC curve for the improved logistic regression model (AUC = 0.605) compared to random guessing.

D. What It Tells Us

Location, timing, and request details carry some information about whether a request will be slow, but not enough to predict it reliably. This suggests that much of what makes a single request slow is not captured in the 311 data itself.

VI. Naive Bayes

A. Why We Chose It

Naive Bayes is a fast, probability-based classifier [5] that answers the same question as logistic regression, which makes it a direct comparison: does a different modeling approach predict slow requests better?

B. Tuning and Training

We used Gaussian Naive Bayes with the same 80/20 split (random state 42), the same features, and the same training-based slow labels as the improved logistic regression model, so the two could be compared fairly.

C. Metrics

Naive Bayes reached 54.1% accuracy and a ROC-AUC of 0.580. For slow requests, precision was 0.60 but recall was only 0.26 (F1 0.36): when it predicted slow it was usually right, but it caught only about a quarter of the truly slow requests (Fig. 4).

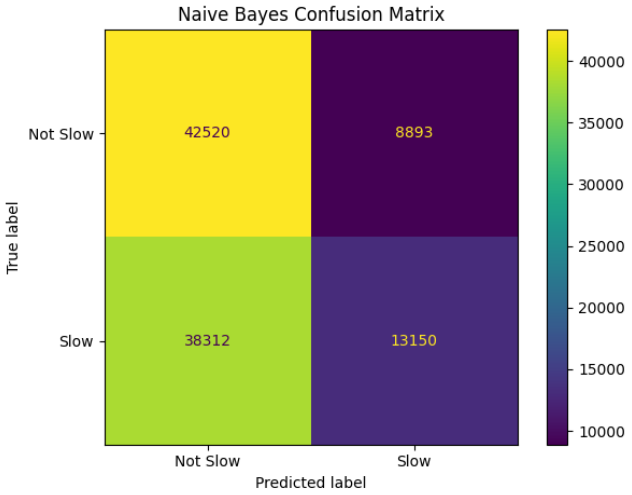


Fig. 4. Naive Bayes confusion matrix on the test set.

D. What It Tells Us

Naive Bayes performed worse than logistic regression on accuracy (54.1% vs. 57.5%) and especially on recall for slow requests (0.26 vs. 0.53). Naive Bayes assumes every feature is independent, but our features overlap heavily (for example, request type and department, or zip code and neighborhood), which likely hurts its predictions. Logistic regression is the better choice of the two for our prediction question.

VII. Random Forest (Outside Class)

A. Why We Chose It

Random Forest was not covered in class; we learned it from the Python Data Science Handbook [5]. It is an ensemble of decision trees [6]: each tree asks a series of yes or no questions and the trees vote, so "the sum can be greater than the parts" [5]. It fits our data for three reasons from the textbook: it is nonparametric and flexible, so it can pick up combinations like district and year without us defining them; training is fast and runs in parallel, which matters with over 500,000 requests; and the trees' votes give a probability for each request. Most

importantly for Q2, its feature importance ranks which factors matter most for slow service.

B. Tuning and Training

We used closed requests only (513,830), the same slow label, and an 80/20 split. The baseline model (district, request type, department, season, and system switch) reached 61% accuracy. The textbook warns that single decision trees overfit easily, and that random forests reduce this by averaging many randomized trees [5]. We tested this by tuning tree depth from 5 to unlimited: accuracy leveled off at about 61% after depth 15, and training and test accuracy stayed within 0.01 of each other at every depth, so the forest did not overfit. Since tuning settings stopped helping, we added the year and month each request was opened, which raised accuracy to 66%. The final model uses 100 trees, a maximum depth of 20, and 77 features.

C. Metrics

The final model reached 66% accuracy compared to 50% for random guessing, with precision 0.65 (fast) and 0.67 (slow), recall 0.69 and 0.63, and F1 0.67 and 0.65. Its predicted slow rate for each district came within about 1 point of the actual rate (Fig. 6), showing its probability estimates are reliable. The model is moderately effective: it beats guessing, trains in seconds, and does notoverfit, but the remaining error shows that factors outside our data also drive slow service.

D. What It Tells Us

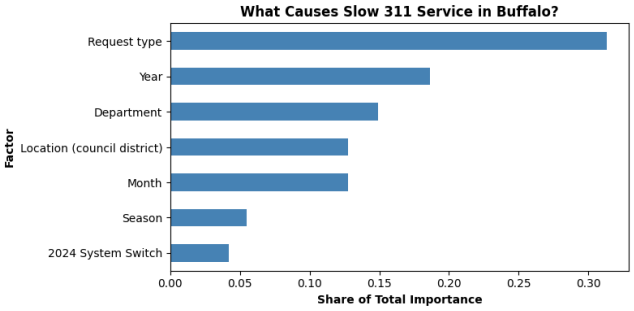


Fig. 5. Random Forest feature importance, grouped by category.

Request type matters most (31%), followed by year (19%), department (15%), location (13%), and month (13%) (Fig. 5). Timing (year, month, season, and system switch) adds up to about 40%. Slow service in Buffalo is explained more by what people request and when than by where they live, which matches Hamel and Holliday's findings across 13 large U.S. cities [7]. Location still matters, though: NORTH's requests were slow 54.5% of the time compared to similar requests, while every other district was between 48% and 51% (Fig. 6). The district and year heatmap (Fig. 7) show predicted slow rates rising in every district from 2020 to 2024, peaking in 2024, the year of the system switch, then improving in 2025 and 2026. One limitation is that random forests are not easily interpretable [5], and importance scores can favor categories with many options, so we treat the ranking as a strong signal rather than an exact measure.

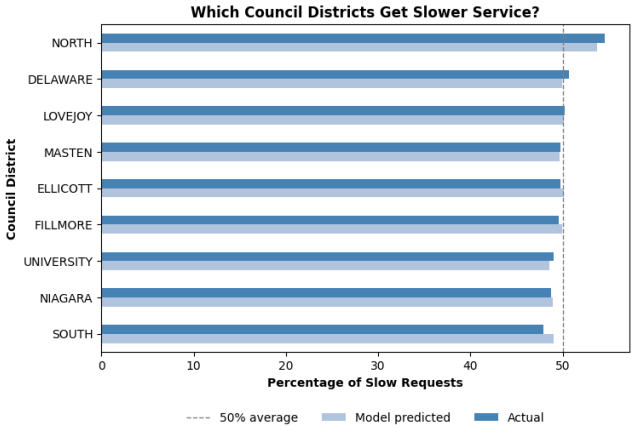


Fig. 6. Predicted and actual share of slow requests by council district, compared within request type.

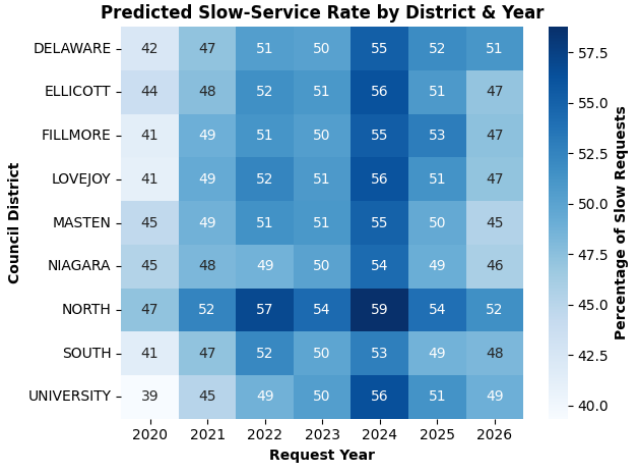


Fig. 7. Random Forest predicted slow-service rate by council district and year.

VIII. K-Means Clustering

A. Why We Chose It

k-Means is an unsupervised algorithm from Lecture 7 [8]: it needs data but no labels, and groups items that are like each other and different from other groups. This fits Q4, since there is no right answer for how neighborhoods should be grouped. We want the data to reveal the groups, then look at what each group has in common.

B. Tuning and Training

We summarized each of Buffalo's 35 neighborhoods with two features: number of 311 requests (demand) and median days to resolve (service speed). Because k-Means groups points by distance, we scaled both with StandardScaler so request counts in the thousands would not outweigh medians of 2 to 5 days. The algorithm assigns each neighborhood to its nearest centroid, moves each centroid to the average of its points, and repeats until nothing changes, minimizing the within-cluster sum of squares J [8]. Using the elbow method from lecture, J dropped sharply up to K = 4 (70, 39, 26, 16) and then flattened (Fig. 8), so we chose K = 4. Since k-Means can converge to a local minimum, we ran it 10 times with different starting centroids (n_init = 10) and kept the best result.

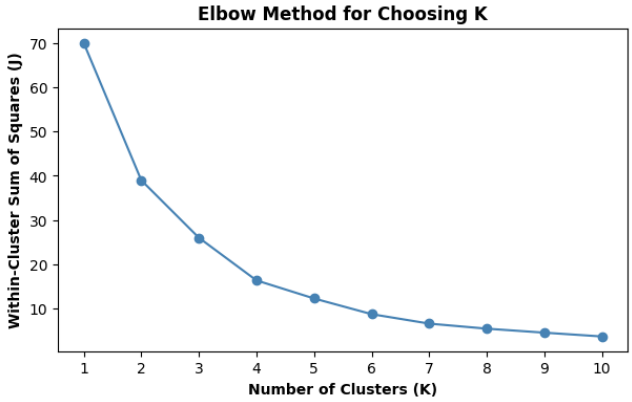


Fig. 8. Elbow method: within-cluster sum of squares (J) for K = 1 to 10.

C. Metrics

We measured cluster quality with the silhouette score, which ranges from -1 to 1 (higher means tighter, better-separated clusters). K = 4 scored 0.459, nearly tied with the best value (K = 5 at 0.468), so the elbow method and silhouette score agree. A score around 0.46 means the clusters reflect real groups, with some neighborhoods near the borders.

D. What It Tells Us

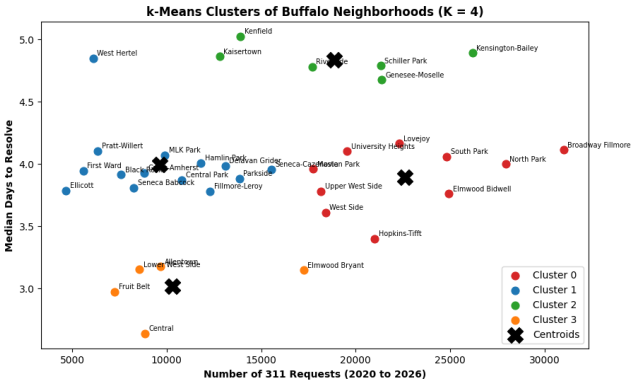


Fig. 9. k-Means clusters of Buffalo neighborhoods (K = 4), with centroids marked.

TABLE III. AVERAGE PROFILE OF EACH NEIGHBORHOOD CLUSTER

Cluster	Hoods	Requests	Median days	% Violations	% Parking
Slowest	6	18,882	4.8	17.9	3.1
Typical	14	9,619	4.0	14.6	4.1
Busiest	10	22,592	3.9	14.9	4.9
Fastest	5	10,317	3.0	10.5	10.9

The four clusters (Fig. 9, Table III) are: the **slowest** group (about 4.8 days), made up of Kenfield, Kaisertown, Riverside, Schiller Park, Genesee-Moselle, and Kensington-Bailey, mostly East Side and Northeast neighborhoods; the **busiest** group (about 22,600 requests, 3.9 days), including Broadway Fillmore, North Park, South Park, and Lovejoy; a **typical** group (about 9,600 requests, 4.0 days); and the **fastest** group (about 3.0 days), made up of Central, Fruit Belt, Allentown, Lower West Side, and Elmwood Bryant, mostly downtown. The slowest neighborhoods have the highest share of housing and building violation requests (17.9%), while the fastest have the lowest (10.5%) and the most parking requests (10.9%), which are usually resolved within a day. Slower neighborhoods wait longer largely because their residents report problems that take longer to fix. One limitation is that results depend on the features we chose: West Hertel has a slow median of 4.8 days but was grouped with the typical cluster because it has very few requests.

IX. DISCUSSION

A. Comparing the Classifiers

TABLE IV. CLASSIFIERS FOR PREDICTING SLOW REQUESTS

Model	Accuracy	Slow recall	Slow F1
Naive Bayes	54.1%	0.26	0.36
Logistic Regression	57.5%	0.53	0.56
Random Forest	66.0%	0.63	0.65

Random Forest performed best (Table IV), though it also used year and month features that the other two did not, so part of its advantage comes from those features. All three beat random guessing, and none predicts individual requests very reliably, which tells us much of what makes a single request slow is not recorded in 311 data.

B. Answering Our Questions

- **Q1 (who waits longest):** By raw medians, UNIVERSITY was the slowest most years from 2022 to 2025. After accounting for request type, NORTH stands out in both linear regression (about 13.5% longer than DELAWARE) and Random Forest (54.5% slow vs. 48% to 51% elsewhere). By neighborhood, Kenfield, Kensington-Bailey, Kaisertown, Riverside, Schiller Park, and Genesee-Moselle wait the longest.
- **Q2 (does request type explain the gaps):** Mostly, yes. Request type alone explains nearly all linear regression's predictive power (R^2 0.277 vs. 0.002 for district), it is Random Forest's top factor (31% vs. 13% for location), and the slowest k-Means cluster has the most housing violation requests. Some district differences, especially NORTH, remain after accounting for type.
- **Q3 (better or worse over time):** Worse through 2024, then slightly better. The district gap roughly doubled from 0.94 days in 2020 to 1.95 days in 2024, and Random Forest's predicted slow rates peaked in every district in 2024, the year of the system switch, before improving in 2025 and 2026.
- **Q4 (shared service patterns):** Neighborhoods fall into four clear groups, and the slowest-served ones share a higher share of housing and building violation requests, while the fastest, mostly downtown, have more quick-to-resolve parking requests.

C. How the Algorithms Differ

Each algorithm answered a different kind of question. Linear regression is a supervised regression model that predicts a

number (days to resolve), while logistic regression, Naive Bayes, and Random Forest are supervised classifiers that predict a category (slow or not). Among the classifiers, logistic regression fits a single weighted equation, Naive Bayes uses probabilities and assumes each feature is independent, and Random Forest combines the votes of many decision trees, which lets it capture combinations of features. MapReduce is not a predictive model but a way to split a large computation (district medians) across many machines. k-Means is unsupervised: it uses no labels and instead finds groups of similar neighborhoods on its own.

X. Limitations

Our models only see what 311 records: crew availability, weather, and the severity of each problem are missing, which limits all our prediction accuracy. Open requests were excluded, and open rates were higher in 2024 and 2026, so recent waits may be underestimated. In Phase 1, missing council districts for 2020 and 2021 were filled from 2011 district lines, which match current boundaries about 90% of the time. Since the data is live, exact counts vary slightly between our notebooks depending on the day each was run.

References

[1] City of Buffalo, "311 Service Requests (July 2008 - May 2024)," Buffalo OpenData. [Online]. Available: https://data.buffalony.gov/Quality-of-Life/311-Service-Requests-July-2008-May-2024-/whkc-e5vr/about_data (accessed Oct. 18, 2026).

[2] City of Buffalo, "311 Service Requests (May 2024 - Present)," Buffalo OpenData. [Online]. Available: https://data.buffalony.gov/Quality-of-Life/311-Service-Requests-May-2024-Present-/3tj7-3tdz/about_data (accessed Oct. 18, 2026).

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th Symp. Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137–150.

[5] J. VanderPlas, *Python Data Science Handbook*. Sebastopol, CA, USA: O'Reilly Media, 2016, ch. 5.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] B. T. Hamel and D. E. Holliday, "Unequal responsiveness in city service delivery: Evidence from 42 million 311 calls," *Quarterly Journal of Political Science*, vol. 19, no. 3, pp. 243–274, 2024, doi: 10.1561/100.00022089.

[8] S. Parvin, "Lec07: Model & algorithm (unsupervised algorithm)," CSE 4/587 Data Intensive Computing, University at Buffalo, Fall 2026.